

cloudtorch / develop — Step 2 outline

A differentiable, 6-parameter-per-pixel background model built on the step-1 PCA basis; supplies I_{in} for step 3. Deliverable of `instructions.md` step 2. All PyTorch. **Model + initialisation only — no optimiser runs here.**

1. OBJECTIVE

Turn the *fixed* step-1 dictionary $\{\boldsymbol{\mu}, \mathbf{V}\}$ ($K = 6$) into a *trainable* background model: each pixel's background is a linear combination of the six eigenprofiles with its own coefficients $\mathbf{c}$. This replaces today's fixed, per-spectrum skglm background (`bdata=new_fit - new_extrafit`) with a compact, differentiable 6-DoF model that step 3 will optimise *jointly* with the two clouds. Step 2 only builds and initialises it.

2. THE MODEL

$$\mathbf{b} = \boldsymbol{\mu} + \mathbf{c} \mathbf{V}, \quad \mathbf{c} \in \mathbb{R}^{n_{\text{pix}} \times 6}, \quad \mathbf{V} \in \mathbb{R}^{6 \times L}, \quad \boldsymbol{\mu} \in \mathbb{R}^L.$$

This is exactly `PCABackground.reconstruct(c)`: linear, autograd-ready, returning (n_{pix}, L) — the incoming intensity I_{in} fed to `cmt.cloud(...)` in step 3. Free parameters: $6 n_{\text{pix}}$ ($= 90,480$ over the 116×130 crop), vs $8 n_{\text{pix}}$ for the two clouds. Keeping $K = 6$ (not 10) deliberately limits the background's freedom to imitate cloud absorption.

3. PARAMETERS & INITIALISATION

- $\mathbf{c}$ is a single `nn.Parameter` of shape $(n_{\text{pix}}, 6)$, following the notebook `make_param` idiom (splittable per-PC later — like the cloud quantities — if freezing is ever wanted).
- Init:** `c0 = pca.project(tofit) = (tofit -  $\boldsymbol{\mu}$ )  $\mathbf{V}^T$` , the orthogonal projection of the *observed* spectra onto the background subspace (reuses `PCABackground.project`).
- Caveat (flag, don't fix):* `tofit` still contains the filament absorption, so `c0` sits slightly low at the absorbed wavelengths. It is only a warm start — step 3 moves $\mathbf{c}$ as the clouds take up the absorption. Fallback: $\mathbf{c} = 0$ ($\boldsymbol{\mu}$ only).

4. INTERFACE & CONSISTENCY

- Returns (n_{pix}, L) ; device/dtype align automatically inside `reconstruct`.
- Tiny module `background_model.py` (mirrors `cloud_model_torch.py`): `init_background_coeffs(pca, obs) → nn.Parameter(npix, 6)`; `background(pca, coeffs) → (npix, L)`.
- Live trap:** `tofit` and the basis must share the *same* λ -crop [100:500] and the *same* `Iqs`. In step 3 the fit's `Iqs` must equal the basis's normalisation, or the background sits at the wrong absolute level and the transfer $I_{\text{out}} = I_{\text{in}} e^{-\tau} + S(1 - e^{-\tau})$ is biased.

5. DEFERRED TO STEP 3 (AND A SANITY CHECK)

- No optimiser in step 2; the joint background+cloud χ^2 + penalties are step 3.
- Optional spatial smoothness on the six coefficient maps via `utils.regu2` — mind its `img.shape[0:2]` reshape, unlike the [1:3] of the other `regu*` — added when we build the composite loss.
- Sanity check** (non-fitting, in the notebook): `residual tofit - reconstruct(project(tofit))`; its map previews the absorption the two clouds must explain, and confirms 6 modes leave the filament — and only the filament — in the residual.

After your review: create `develop/background_model.py` and a step-2 test cell (init `c` from `tofit`, plot background vs observed, residual map, confirm `coeffs.grad`). Then step 3 — composite background + two clouds.